



Implementasi Metode Simple Additive Weighting (SAW) Berbasis Pemrosesan Paralel Menggunakan MPI

Studi Kasus: Rekomendasi Destinasi Wisata Skala Besar

Tugas Besar Mata Kuliah
Pemrosesan Paralel (2026)

Dosen Pengampu:
Heri Wijayanto, S.T., M.T., Ph.D.

Disusun Oleh:
Alya Rasyida Putri (F1D02410002)
Sabrina Mawadathun Salsabila (F1D02410092)
Felyvia Trevina Gunawan (F1D02410112)
Nadine Desvita Alifia Putri (F1D02410129)

Latar Belakang & Permasalahan



Pentingnya SPK

Sistem Pendukung Keputusan (SPK) membantu penentuan alternatif secara sistematis, objektif, dan terstruktur.



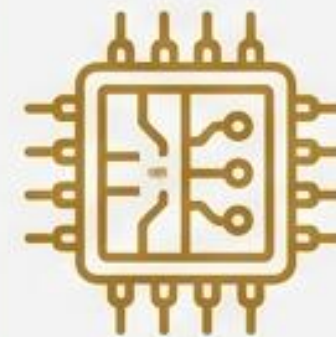
Tantangan Eksekusi

Banyaknya alternatif destinasi wisata menyulitkan pengguna dalam mengambil keputusan secara tepat dan akurat.



Kebutuhan Big Data

Evaluasi jutaan alternatif data membutuhkan komputasi tinggi, memicu waktu eksekusi yang lambat jika diproses secara sekuensial.



Solusi Integrasi (SAW + MPI)

SAW menyeleksi alternatif terbaik secara matematis dan objektif. MPI mendistribusikan beban komputasi secara paralel untuk mempercepat waktu eksekusi secara drastis.

Tujuan dan Manfaat Penelitian



Tujuan Penelitian

1. Menerapkan metode SAW untuk penentuan destinasi wisata dan menganalisis hasil perangkingan secara sistematis.
2. Mengintegrasikan arsitektur komputasi paralel untuk menangani kendala beban pemrosesan data.



Manfaat SAW (Akurasi)

1. Mengurangi subjektivitas dalam proses pemilihan destinasi wisata.
2. Memberikan rekomendasi Top-N alternatif terbaik secara objektif berdasarkan kriteria dinamis.



Manfaat MPI (Efisiensi)

1. Mempercepat proses perhitungan pada dataset berukuran raksasa (Big Data).
2. Mendistribusikan beban kerja komputasi secara merata ke berbagai prosesor (node).

Dasar Teori: Integrasi Algoritma & Infrastruktur



Dataset & Kriteria Penilaian (Atribut)

Sistem mengevaluasi jutaan baris data menggunakan 6 kriteria utama pembentuk scoring model.

Atribut	Bobot	Sifat Kriteria
Visitors (Pengunjung)	25%	↗ Benefit
Rating (Penilaian)	20%	↗ Benefit
Revenue (Pendapatan)	20%	↗ Benefit
Acc. Availability (Fasilitas)	15%	↗ Benefit
Distance (Jarak)	10%	↓ Cost
Ticket Price (Harga Tiket)	10%	↓ Cost

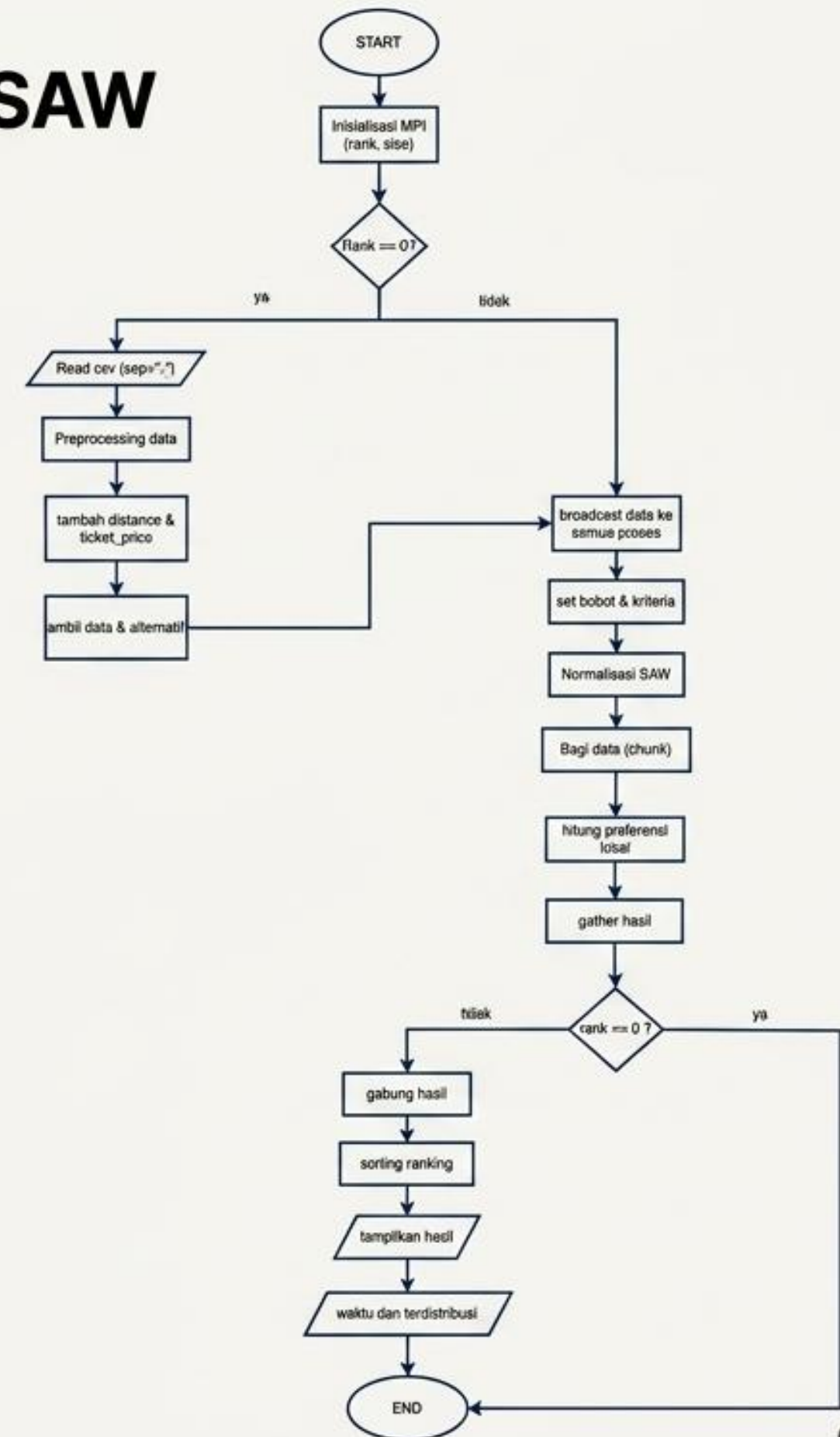
Atribut Benefit (Keuntungan)

Semakin besar nilai atribut, semakin baik hasil preferensinya.
(Normalisasi = Nilai dibagi dengan Nilai Maksimal).

Atribut Cost (Biaya)

Semakin kecil nilai atribut, semakin baik hasil preferensinya.
(Normalisasi = Nilai Minimal dibagi dengan Nilai).

Metodologi Sistem: Alur Kerja Algoritma SAW



Implementasi Paralel Menggunakan MPI

1. Broadcast Data (comm.bcast)

Master Process mengirimkan dataset, kriteria, dan bobot secara identik ke semua node.

2. Data Chunking

Dataset masif dipecah secara merata (chunk = rows // size) ke masing-masing memori prosesor.

3. Perhitungan Paralel (Lokal)

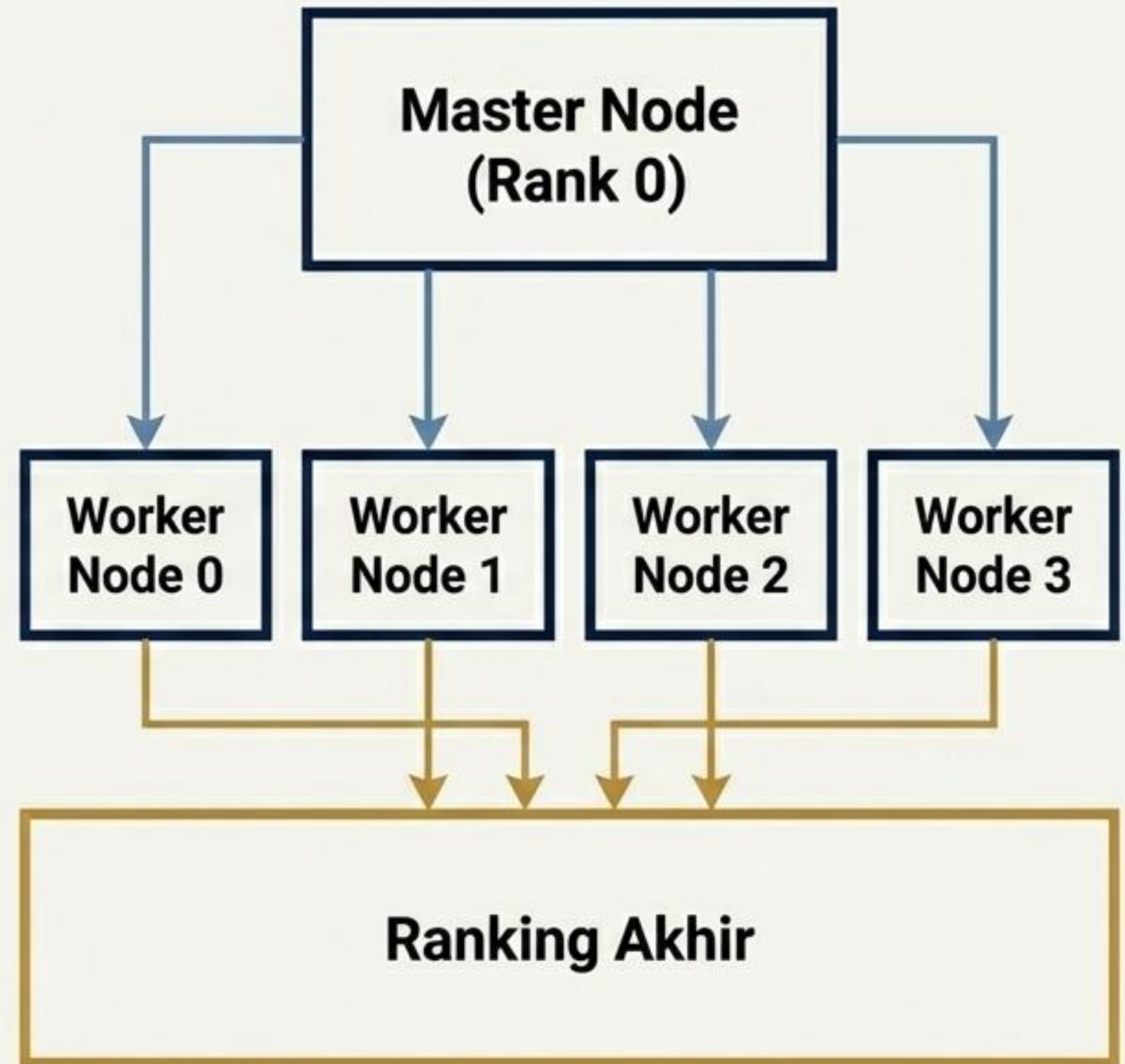
Setiap node mengeksekusi perhitungan preferensi SAW murni secara independen pada porsi datanya.

4. Gather Hasil (comm.gather)

Seluruh matriks skor lokal disinkronisasi dan ditarik kembali ke Master Process.

5. Sorting & Ranking

Pengurutan nilai tertinggi memunculkan rekomendasi wisata teratas.



Hasil Pengujian Sistem (Skala 300.000 Data)

Eksekusi 2 Proses (Node)

```
PS C:\Users\asus\OneDrive\Documents\pemparr> mpiexec - python mpi_baru.py

=== RANKING SEMUA DATA ===

1. Lombok Beach (Brazil) - Beach -> 0.9281
2. Osaka Castle (Brazil) - Cultural -> 0.9111
3. Nlagara Falls (USA) - Nature -> 0.9052
4. Guilin Karst (China) - Urban -> 0.9849
5. Mont Saint-Nichel (France) - Historical -> 0.8969
6. Berlin Wall (France) - Beach -> 0.8949
7. Niagara Falls (USA) - Beach -> 0.8935
8. Bangkok Temples (USA) - Cultural -> 0.8868
9. Mount Fuji (USA) - Nature -> 0.8854
18. Lake Toba (France) - Historical -> 0.8554

Maktu eksekusi: 2.4835253000001103 detik
Total data: 300000

Distribusi proses:
Proses 0 mengerjakan 150000 data
Proses 1 mengerjakan 150000 data
Proses 1 mengerjakan 150000 data
```

Waktu Eksekusi: ~2.48 detik
Distribusi: Masing-masing prosesor
menghitung tepat 150.000 baris data.

Eksekusi 4 Proses (Node)

```
PS C:\Users\asus\OneDrive\Documents\pemparr> mpiexec - 4 python mpi_baru.py

=== RANKING SEMUA DATA ===

1. Phi Phi Islands (India) - Beach -> 0.9492
2. Shanghai Bund (China) - Beach -> 0.9252
3. Langkawi Island (India) - Beach -> 0.9239
4. Seoraksan Park (Australia) - Nature -> 0.9155
5. Amazon Rainforest (Brazil) - Nature -> 0.9135
6. Edinburgh Castle (France) - Adventure -> 0.9124
7. Kyoto Gardens (USA) - Cultural -> 0.9122
8. Famous Landmark (Egypt) - Beach -> 0.9093
9. Raja Aempat (India) - Nature -> 0.9884
10. Bromo Volcano (Brazil) - Adventure -> 0.9010

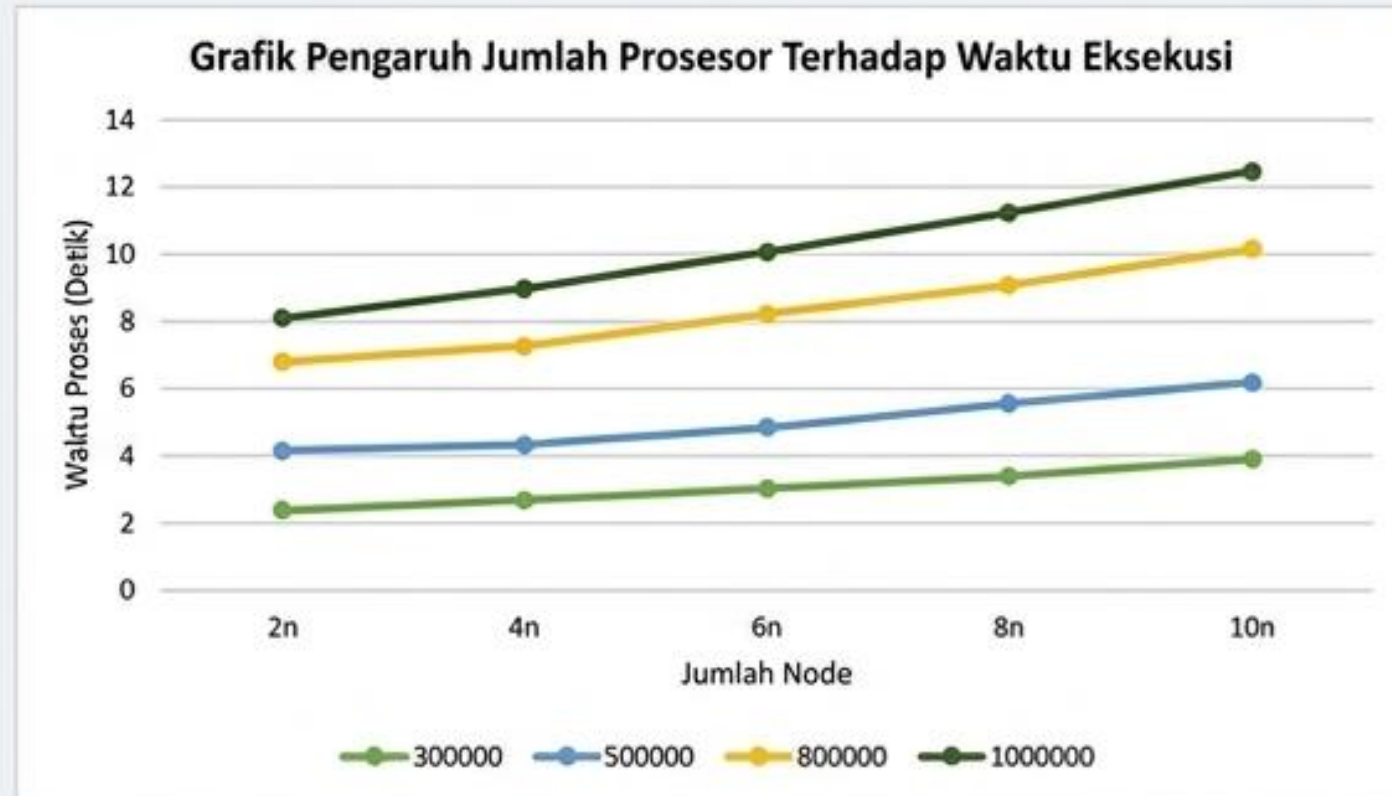
Maktu eksekusi: 2.6328914999994595 detik
Total data: 300000

Distribusi proses:
Proses 0 mengerjakan 75000 data
Proses 1 mengerjakan 75000 data
Proses 2 mengerjakan 75000 data
Proses 3 mengerjakan 75000 data
```

Waktu Eksekusi: ~2.63 detik
Distribusi: Keempat prosesor menghitung
tepat 75.000 baris data secara bersamaan.

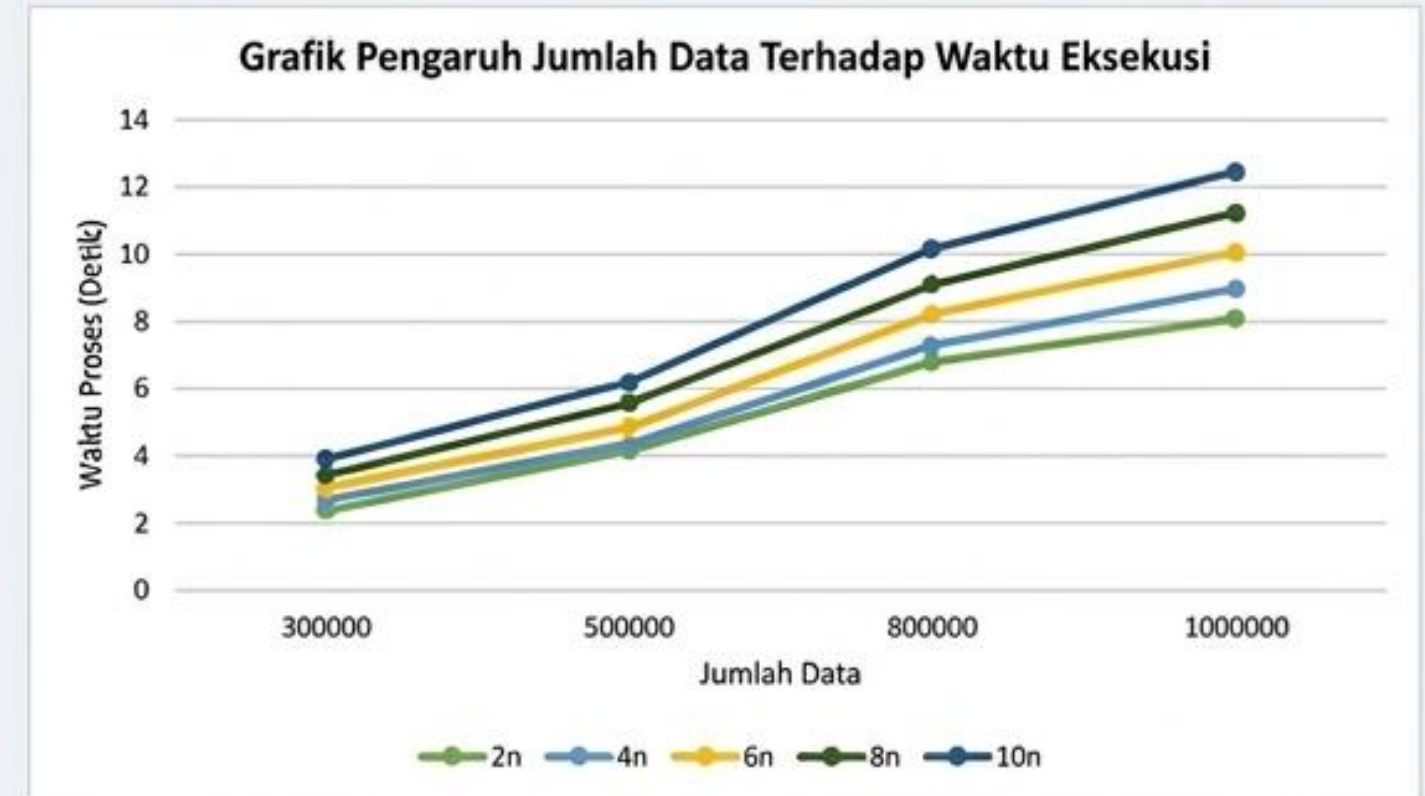
Sistem konsisten mendeteksi 'Phi Phi Islands' dan 'Lombok Beach' sebagai Top Rekomendasi, membuktikan akurasi matematis tidak berubah saat dipecah paralel.

Analisis Performa: Korelasi Data, Node, dan Waktu



Pengaruh Jumlah Prosesor

- **Temuan Kritis:** Menambah node (dari 2n ke 10n) tidak selalu mempercepat komputasi.
- **Penyebab:** Munculnya 'Communication Overhead', di mana waktu sinkronisasi pesan antar node lebih dominan dibandingkan waktu kalkulasi murni SAW.



Pengaruh Skala Dataset

- **Temuan Kritis:** Waktu eksekusi berbanding lurus secara linier dengan volume data.
- **Skalabilitas:** MPI tetap stabil dan tidak mengalami crash meskipun menangani matriks keputusan hingga 1 Juta baris secara konstan.

Kesimpulan Analisis: Optimasi rasio pembagian beban vs overhead komunikasi sangat krusial untuk efisiensi paralel tertinggi.

Kesimpulan dan Rekomendasi

Kesimpulan Akhir



Akurasi SAW Terbukti: Algoritma sukses berfungsi secara objektif dan sistematis mengolah scoring model dengan 6 kriteria berbeda.



MPI Mengeliminasi Bottleneck: Proses data chunking berhasil memangkas waktu eksekusi sekuensial; memproses 300.000 matriks dalam ~2.5 detik.



Skalabilitas Enterprise: Infrastruktur paralel secara stabil mampu memproses hingga 1.000.000 baris data wisata tanpa degradasi memori.

Saran Pengembangan



Komparasi Metodologi: Menguji hasil matriks SAW dan membandingkannya dengan metode SPK lain (TOPSIS, AHP, atau WP).



Reduksi Overhead Jaringan: Mengimplementasikan Non-Blocking Communication pada MPI untuk menurunkan waktu jeda sinkronisasi.



Integrasi Tampilan Visual: Mengangkat arsitektur ini ke backend berbasis Web atau Mobile (UI/UX) untuk utilitas pengguna akhir.